

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: امیر پارسا علیزاده

شماره دانشجویی: 02220040709015

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

گزارش تحلیل کد: شبیه‌ساز گیت مترو

۱. معرفی کلی پروژه

این برنامه یک شبیه‌ساز گرافیکی گیت ورودی/خروجی مترو است که با استفاده از کتابخانه tkinter برای رابط کاربری پیاده‌سازی شده است. ساختار کد بر پایه‌ی برنامه‌نویسی شیء‌گرا (OOP) طراحی شده و هر بخش از سیستم واقعی (کارت، مسافر، سنسور، موتور، گیت) به‌صورت یک کلاس مجزا مدل‌سازی شده است. این رویکرد باعث می‌شود کد از انسجام بالا و وابستگی کم بین اجزا (Low Coupling / High Cohesion) برخوردار باشد.

۲. دسته‌بندی بخش‌های کد

۲.۱. لایه‌ی منطق تجاری (Business Logic)

شامل کلاس‌های Card، Gate، Motor، Sensor، Passenger، ::

۲.۲. لایه‌ی رابط کاربری (Presentation Layer)

شامل کلاس GUI: و متدهای زیرمجموعه‌ی آن برای ساخت المان‌ها، مدیریت رویدادها و انیمیشن

۲.۳. نقطه‌ی اجرای برنامه (Entry Point)

بلاک: `if __name__ == "__main__":`

این جداسازی، هرچند کامل نیست چون منطق و UI هر دو در یک فایل قرار دارند اما از نظر مفهومی، الگوی طراحی MVC ساده‌شده را دنبال می‌کند.

۳. تحلیل کلاس‌های منطق تجاری

۳.۱. کلاس Card

```
def has_credit(self, cost=5000):  
    return self.balance >= cost
```

```
def charge(self, cost=5000):  
    if self.has_credit(cost):  
        self.balance -= cost  
        self.history.append(...)  
        return True  
    return False
```

جداسازی متد `has_credit` از `charge` نمونه‌ای از اصل مسئولیت واحد (Single Responsibility Principle) است؛ بررسی اعتبار از عملیات کسر مبلغ جدا شده تا امکان استفاده‌ی مستقل از هرکدام (مثلاً فقط بررسی بدون کسر) فراهم باشد. همچنین ثبت `history` نوعی **Audit Log** ساده برای ردیابی تراکنش‌هاست.

۳.۲. کلاس `Passenger`

```
class Passenger:
    _next_id = 1
    def __init__(self, name, card=None):
        self.id = Passenger._next_id
        Passenger._next_id += 1
```

استفاده از یک متغیر سطح کلاس (Class Variable) به‌عنوان شمارنده‌ی خودکار برای تولید شناسه‌ی یکتا- (Auto-increment ID)، مشابه کلید اصلی (Primary Key) در پایگاه داده.

همچنین متد `__str__` بازنویسی شده تا نمایش شیء در `Combobox` رابط کاربری خوانا باشد

۳.۳. کلاس `Motor`

```
class Motor:
    STATES = ("closed", "opening", "open", "closing")
```

این کلاس عملاً یک ماشین حالت محدود (Finite State Machine - FSM) ساده را پیاده‌سازی می‌کند. متدهای `start_opening`, `finish_opening`, `start_closing`, `finish_closing` مجاز بین حالات را کنترل می‌کنند و ویژگی `is_open` با دکوراتور `@property` به‌صورت فقط-خواندنی (read-only) در دسترس قرار می‌گیرد. این طراحی از تغییر مستقیم و ناخواسته‌ی وضعیت جلوگیری می‌کند.

۳.۴. کلاس `Gate`

این کلاس نقش هماهنگ‌کننده (Coordinator/Facade) بین `Sensor`، `Motor` و `Card` را دارد:

```
def request_entry(self, passenger):
    if passenger.card is None: ...
    if not passenger.card.has_credit(...): ...
    passenger.card.charge(...)
    self.sensor.detect(True)
    self.motor.start_opening()
```

ترتیب فراخوانی‌ها (ابتدا بررسی اعتبار، سپس کسر وجه، سپس فعال‌سازی سنسور و موتور) نشان‌دهنده‌ی رعایت قاعده‌ی **Fail-Fast** است.

متد `request_exit` بدون هیچ بررسی اعتبارسنجی، همیشه `True` برمی‌گرداند.

۴. تحلیل کلاس GUI

۴.۱. ثابت‌های سطح کلاس برای مختصات

```
LEFT_POST = (90, 20, 110, 180)
BAR_CLOSED = (110, 90, 290, 110)
BAR_OPEN = (90, 90, 110, 110)
```

این مقادیر به جای Magic Numbers پراکنده در کد، به صورت ثابت‌های نام‌گذاری شده در سطح کلاس تعریف شده‌اند که خوانایی و قابلیت نگهداری کد را افزایش می‌دهد.

۴.۲. متد `_run_animation` — بازگشتی غیرمسدودکننده (Non-blocking Recursive Animation)

```
def _run_animation(self, closing, on_finish, step=0):
    ...
    if step < self.ANIMATION_STEPS:
        self.root.after(self.ANIMATION_DELAY_MS,
                        lambda: self._run_animation(closing, on_finish, step + 1))
    else:
        on_finish()
```

به جای استفاده از حلقه‌ی `for` با `time.sleep()` که رابط کاربری Tkinter را در طول اجرا فریز (**Freeze**) می‌کند از متد `root.after()` برای زمان‌بندی فراخوانی بعدی تابع استفاده شده است. این تکنیک معادل **Callback-based Animation Loop** است و اصل حیاتی برنامه‌نویسی رویدادمحور (Event-driven Programming) در GUI را رعایت می‌کند: هرگز نباید Thread اصلی (UI Thread) را مسدود کرد.

پارامتر `on_finish` نیز نمونه‌ای از **Higher-order Function** است؛ رفتار پایانی انیمیشن (باز شدن یا بسته شدن) به صورت تابعی به متد عمومی انیمیشن تزریق می‌شود که از تکرار کد جلوگیری می‌کند اصل DRY

۴.۳. متد استاتیک `_interpolate`

```
@staticmethod
def _interpolate(start, end, ratio):
    return tuple(s + (e - s) * ratio for s, e in zip(start, end))
```

این متد درونیابی خطی (**Linear Interpolation / Lerp**) را بین دو مجموعه مختصات (حالت باز و بسته‌ی نوار گیت) محاسبه می‌کند. استفاده از `@staticmethod` صحیح است چون تابع به هیچ‌یک از ویژگی‌های نمونه (`self`) نیاز ندارد

۴.۴. مدیریت بستن خودکار گیت با after و after_cancel

```
self._auto_close_job = self.root.after(self.AUTO_CLOSE_DELAY_MS, self.animate_close)
...
def _cancel_auto_close(self):
    if self._auto_close_job is not None:
        self.root.after_cancel(self._auto_close_job)
```

این بخش یک مکانیزم تایمر قابل لغو (Cancellable Timer) است. ذخیره‌ی شناسه‌ی Job بازگشتی از after() و امکان لغو آن با after_cancel() از بروز باگ رایج جلوگیری می‌کند: بدون این مکانیزم، اگر کاربر گیت را به صورت دستی ببندد، تایمر خودکار قبلی همچنان در پس‌زمینه اجرا شده و باعث رفتار غیرمنتظره (مثلاً بسته شدن دوباره‌ی گیتی که به تازگی باز شده) می‌شود.

۴.۵. جداسازی متدهای *_build_

تفکیک ساخت رابط کاربری به متدهای مجزا (_build_canvas, _build_passenger_controls, _build_action_buttons, _build_log_area) نمونه‌ای از اصل **Separation of Concerns** در طراحی رابط کاربری است و از تبدیل شدن __init__ به یک متد طولانی و غیرقابل نگهداری جلوگیری می‌کند.